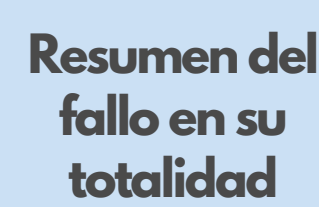
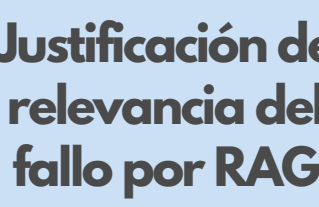
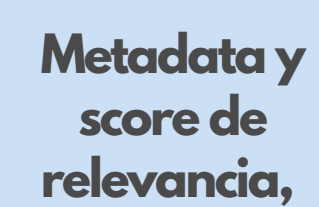
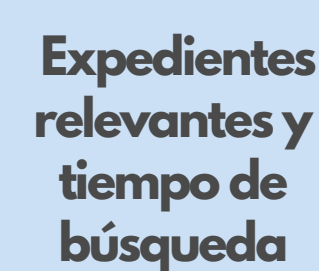


Optimizar la búsqueda judicial mediante una herramienta inteligente que, ante cada consulta, ofrece automáticamente fragmentos relevantes de fallos anteriores, un resumen con la idea central y las fuentes citadas.

Query inicial



Etapa 1		Etapa 2		Etapa 3		Etapa 4		Etapa 5		Etapa 6	
Recolección de Datos		Procesamiento		Generación de Features		Almacenamiento		RAG		Interfaz y Deployment	
Contactar con el Superior Tribunal de Justicia de Entre Ríos		Conversión PDF a TXT		LLM - Listar de artículos citados		Chunking y Embeddings		Arquitectura Híbrida		Creación de API	
		Limpieza de archivos		LLM - Idea central del fallo		Almacenamiento Base de Datos Vectorial		Scoring		Generación de Frontend	
		División de secciones		Armado de JSON		Almacenamiento Compactado para BM25		ReRanking		Dockerizar el proyecto	
		Extracción de Metadatos						Prompting LLM			

The diagram illustrates a hybrid search architecture. It starts with a 'Consulta' (Query) input, which is processed by two parallel paths: 'Embedding' leading to a 'Buscador Denso (Qdrant)' and 'Tokenizar' leading to a 'Buscador Lexico (BM25)'. The results from these searches are ranked ('Top n' and 'Top m') and then combined in a 'Scoring (Dense - Lexico - Metadata + Consulta)' step. This scoring step also receives feedback from the 'Response' block. The output of the scoring is then passed to a 'ReRank (CrossEncoder)' block, which finally outputs 'Top K' results to the 'LLM' (Large Language Model) block. The 'LLM' block then generates the final 'Response'.

Combina búsqueda semántica (embeddings + Qdrant) y coincidencia léxica (BM25), refuerza con metadatos de resultados con la pregunta del usuario y rerankea con Cross-Encoder para devolver los párrafos más relevantes.

Contamos con un dataset de fallos del Superior Tribunal de Justicia de Entre Ríos, Sala Civil y Comercial

- Contamos con todos los fallos de la sala entre las fechas 02/2024 y 12/2024
- 296 fallos
 - 93 Auto
 - 203 Sentencia
- Sobre las Sentencias:
 - Promedio de 12 páginas
 - Promedio de 2800 palabras
 -

Generador de Idea Central

Resumen automatizado del contenido jurídico:

- Dado que los fallos variaban entre 1 y 25 páginas, implementamos un generador de idea central para condensar su contenido.
- Utilizamos GPT-4o con prompting few-shot, orientado a capturar el núcleo argumental de cada fallo en menos de 300 tokens.
- Este resumen facilitó el análisis posterior al preservar la esencia jurídica en un formato compacto y procesable.

Extractor de Referencias Citadas

Paso 1: Extracción y normalización de referencias legales

- Usamos GPT-4o con prompting few-shot para identificar referencias legales citados en cada uno de los fallos.
- El resultado de este análisis arrojó 126 referencias con alta redundancia y sin estandarización (ej. “Ley nro x”, “CCCN”, etc.).
- Un segundo pasaje por el modelo permitió estandarizar las citas, reduciendo a 55 fuentes únicas.

Paso 2: Extracción estructurada con esquema validado

- A partir de las 55 referencias estandarizadas, definimos un esquema anidado en Pydantic con campos de selección restringida.
- Reprocesamos los fallos con GPT-4o, alimentando el modelo con el esquema como guía.
- Obtuvimos un JSON estandarizado por fallo, con citas normalizadas y validadas automáticamente
- El proceso resultante es más robusto y reproducible.

Limpieza de datos:

- Recolección de fallos de una cámara federal.
- Normalización de archivos PDF y eliminación de texto irrelevante.
 - Se eliminaron las secciones irrelevantes de cada uno de los PDF (Márgenes con información irrelevante, numeración de páginas, etc.)
- Conversión a formato TXT.
- Separación del texto en secciones e identificación de la “Materia Preliminar” de cada fallo.
- Armado de archivos JSON para estructurar cada fallo de forma estandarizada.

Chunking y Embeddings

Separado del texto en chunks considerando lo siguiente:

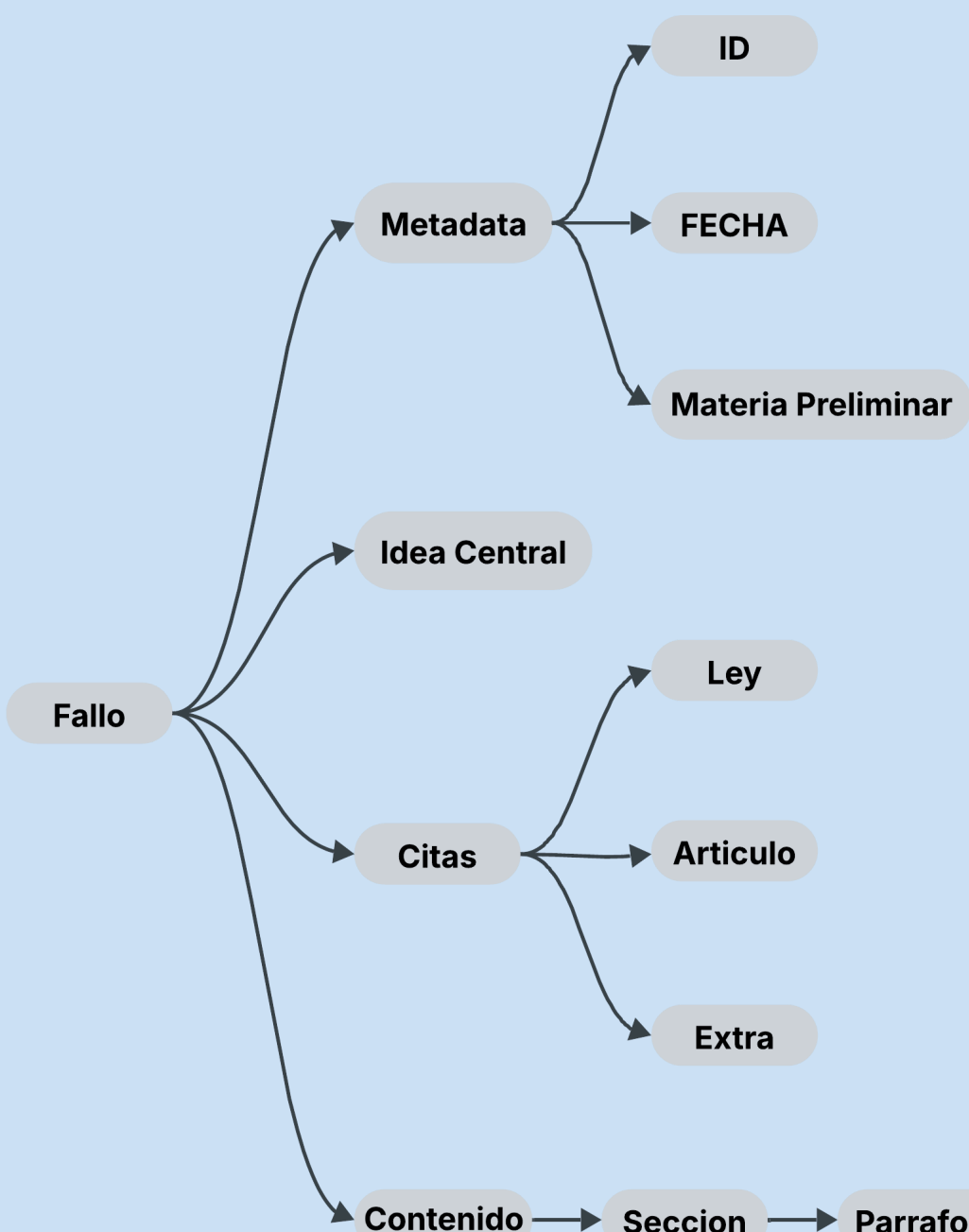
- Respetar las secciones
- Un chunk por párrafo
- Metadatos para mantener contexto

Cálculo de Embeddings

- Usa sentence-transformers/all-MiniLM-L6-v2
- embeddings de 384 dimensiones
- Salida directa a Qdrant
- Compatible con búsqueda híbrida BM25

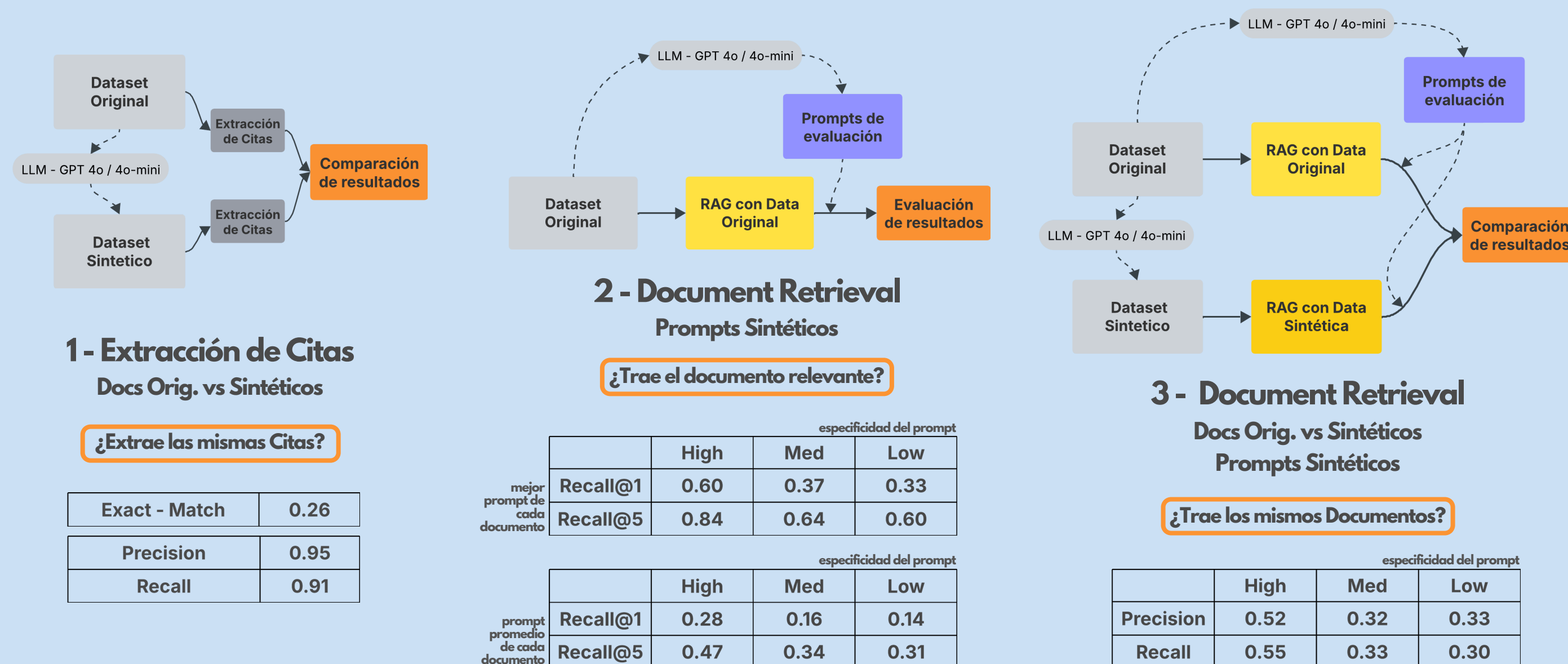
Estructura del JSON

Mantener una estructura homogénea permite reutilizar la información en todo el proyecto, desde la incorporación y evaluación de nuevos Features hasta el entrenamiento de modelos. Cada vez que ingresa un fallo, basta con generar su JSON una única vez.



Evaluación

El sistema fue evaluado con cuatro pruebas: tres experimentos controlados con datos y prompts sintéticos, que midieron robustez y precisión/consistencia de la recuperación de documentos, y una prueba de uso real con un juez.



4 - Test de Campo - Evaluación con Juez Real

Puntos Positivos

- Consideró “Muy Buenos” los campos que trae la respuesta del RAG

Puntos críticos

- Para algunas preguntas, considera que estaría bueno agregar el modo 'conversacional'
- Hubo algunos temas particulares en los que no se obtuvieron los fallos esperados.